PATHPILOT

From an interactive grid game to A* optimization and a supervised CNN navigator

Engineering guide through Phase 3, Milestone 5

Architecture | Code walkthroughs | Testing | Benchmarks | ML pipeline

Current snapshot

107 automated tests pass. Phase 1 and Phase 2 are complete. Phase 3 includes dataset generation, a 29,988-parameter CNN, training/checkpointing, cached inference, and Pygame integration. Formal held-out navigation evaluation is the next milestone.

How to use this guide

This document explains what was built, why each design was chosen, how the major code paths work, and what the measured results mean. It is organized chronologically so a reader can understand PathPilot without already knowing Pygame, A*, or PyTorch.

Contents

Part	Focus
1. System overview	Repository layout, separation of concerns, and shared data flow
2. Phase 1	Grid environment, Pygame UI, controls, and manual navigation
3. Phase 2	A*, visualization, baseline vs cached controller, and benchmarks
4. Phase 3	Teacher dataset, CNN, training, checkpointing, inference, and UI integration
5. Verification	Commands, tests, measured results, limitations, and next milestone

> **Important interpretation**
>
> Action-classification accuracy is not the same as navigation success. A model can predict many individual steps correctly and still fail an episode after one wrong turn. PathPilot intentionally records these separately.

Guiding engineering rules

- The environment is independent of Pygame, A*, and PyTorch.
- Every agent uses the same movement and collision rules.
- Generated data and checkpoints are reproducible from saved seeds and configuration.
- Optimization claims require measured benchmarks.
- Each phase adds one capability without rewriting earlier layers.

1. System architecture

PathPilot is organized as a layered system. The grid environment is the stable core; interfaces and agents sit above it. This makes it possible to compare manual control, A*, and neural policies under identical rules.

Repository map

```
src/pathpilot/
|-- environment.py          # Grid rules and mutable state
|-- game.py                 # Event loop and coordination
|-- ui/                     # Layout and Pygame rendering
|-- planning/               # Planner protocol and A*
|-- controllers/            # A* and CNN navigation policies
|-- metrics/                # Run measurements
|-- evaluation/             # Seeded scenarios and benchmarks
`-- learning/               # Dataset, CNN, training, inference

tests/                      # Mirrors important source behavior
benchmarks/                 # Phase 2 reference results
datasets/                   # Generated and ignored by Git
artifacts/                  # Trained models and histories, ignored by Git
```

Why this separation matters

Layer	Knows about	Does not know about
Environment	Grid coordinates, endpoints, obstacles, movement	Pygame, A*, tensors, models
Pygame UI	Pixels, buttons, colors, keyboard and mouse	Pathfinding implementation details
Controllers	When and how an agent acts	Rendering details
Learning	Encodings, labels, tensors, weights	Mouse and window events

2. Phase 1 - Playable navigation game

Phase 1 established a complete interactive game before any AI was added. Users can paint and erase obstacles, place start and goal cells, move manually, reset only the agent, or clear the board.

The environment model

```
class Direction(Enum):
    UP = (0, -1)
    DOWN = (0, 1)
    LEFT = (-1, 0)
    RIGHT = (1, 0)

class GridEnvironment:
    def move(self, direction: Direction) -> MoveResult:
        dx, dy = direction.value
        destination = (self.agent[0] + dx, self.agent[1] + dy)
        if not self.in_bounds(destination):
            return MoveResult.OUT_OF_BOUNDS
        if destination in self.obstacles:
            return MoveResult.BLOCKED_BY_OBSTACLE
        self.agent = destination
        return MoveResult.REACHED_GOAL if self.reached_goal else MoveResult.MOVED
```

The enum expresses legal one-cell deltas. **move()** calculates a candidate cell, rejects boundaries and obstacles, then mutates the agent only for a valid move. Returning a **MoveResult** lets the UI explain exactly what happened without duplicating rules.

> ### Key design decision
>
> Start, goal, obstacles, and agent position live in GridEnvironment. Pygame renders that state but does not own it. A future AI can therefore call the same move() method as a human player.

Safe editing

- Obstacles cannot cover the start, goal, or current agent.
- Clicking an existing obstacle toggles it back to an empty cell.
- Left-drag paints obstacles; right-click or right-drag erases them.
- Selecting a new start also returns the agent to that start.

Phase 1 UI and Pygame concepts

Pygame uses a continuous event loop. Each frame reads input events, updates state, draws the current state, and flips the display buffer.

```
while running:
    elapsed_ms = clock.tick(60)
    for event in pygame.event.get():
        if event.type == pygame.KEYDOWN:
            # Convert a key into a Direction.
            environment.move(key_directions[event.key])
        elif event.type == pygame.MOUSEBUTTONDOWN:
            # Hit-test buttons or convert pixels to a grid cell.
            position = ui.grid_position_at(event.pos)

    ui.draw(environment, active_tool, status, pygame.mouse.get_pos())
    pygame.display.flip()
```

clock.tick(60) limits rendering to approximately 60 frames per second and returns elapsed milliseconds. Event objects describe keyboard, mouse, and window actions. **pygame.display.flip()** reveals the fully rendered next frame at once, preventing partially drawn visuals.

Pixel-to-grid conversion

```
def grid_position_at(layout, point):
    x, y = point
    if point_is_outside_board:
        return None
    return (
        (x - layout.grid_left) // layout.cell_size,
        (y - layout.grid_top) // layout.cell_size,
    )
```

The layout module performs geometry and hit-testing without importing Pygame. Subtracting the board origin converts window pixels into local coordinates; integer division selects the column and row.

Phase 1 controls

Control	Behavior
Obstacles	Toggle with click, paint with left-drag, erase with right-drag
Set Start / Set Goal	Select a mode, then click a valid cell
Play	Use arrows or WASD with collision feedback
Reset Agent	Return to current start without clearing obstacles
Clear Grid	Restore original endpoints and remove obstacles

3. Phase 2 - A* pathfinding

Phase 2 added a deterministic shortest-path planner. A* prioritizes cells using the distance already traveled plus an estimate of the distance remaining.

A* priority

f(n) = g(n) + h(n). Here g(n) is the known movement cost from the start, and h(n) is Manhattan distance to the goal.

```
class AStarPlanner:
    @staticmethod
    def heuristic(position, goal):
        return abs(position[0] - goal[0]) + abs(position[1] - goal[1])

    def plan(self, environment, start, goal):
        frontier = []
        heapq.heappush(frontier, (self.heuristic(start, goal), 0, start))
        came_from = {start: None}
        cost_from_start = {start: 0}

        while frontier:
            _, _, current = heapq.heappop(frontier)
            if current == goal:
                return self._reconstruct_path(came_from, current)
            for neighbor in environment.neighbors(current):
                tentative_cost = cost_from_start[current] + 1
                # Keep a cheaper route and push its estimated total cost.
```

A heap-based priority queue retrieves the most promising cell efficiently. **came_from** stores parent links for path reconstruction, while **cost_from_start** prevents more expensive duplicates from replacing cheaper routes.

Planner result contract

- path includes both requested start and goal; an empty path means failure.
- explored_cells preserves search order for animation.
- nodes_explored and planning_latency_ns support benchmarks.
- The planner never mutates the environment.

Visualizing search

The A* planner runs immediately and returns exploration history. A separate visualization controller reveals that history over time. This keeps animation timing out of the search algorithm.

Timed controller lifecycle

```
def update(self, elapsed_ms, environment):
    if self.state is not NavigationState.RUNNING:
        return
    self._elapsed_ms += elapsed_ms
    if self._elapsed_ms < self.movement_interval_ms:
        return
    self._elapsed_ms %= self.movement_interval_ms
    self._advance(environment)
```

Controllers accumulate frame time and act only after their configured interval. The game remains responsive because there is no blocking sleep call.

Dynamic obstacles

Obstacle editing remains enabled during autonomous navigation. The V1 controller sees every edit because it replans before every step. V2 checks whether a changed obstacle intersects the remaining cached route and replans only when necessary.

Separation of time scales

Rendering can run at 60 FPS while exploration or movement advances more slowly. The controller receives elapsed time instead of owning the main loop.

V1 versus V2 navigation

Both controllers use the same A* implementation and movement rules. The difference is when each controller chooses to call the planner.

Strategy	Behavior	Expected computation
V1 Always-Replan	Discard the path after one move and run A* again	One planning call per movement
V2 Cached-Path	Reuse the route until it is exhausted or blocked	One initial call plus necessary replans

V2 decision

```
def _needs_plan(self, environment):
    if len(self._remaining_path) < 2:
        return True
    if self._remaining_path[0] != environment.agent:
        return True
    return any(
        position in environment.obstacles
        for position in self._remaining_path[1:]
    )
```

V2 replans when it has no usable route, when the route no longer begins at the current agent, or when any future route cell becomes blocked. An obstacle behind the agent or away from the route does not trigger unnecessary work.

Metrics

```
@dataclass
class NavigationMetrics:
    planning_latency_ns: int = 0
    nodes_explored: int = 0
    planning_calls: int = 0
    replanning_count: int = 0
    path_length: int = 0
```

Phase 2 reproducible benchmark

The benchmark uses seeded maps and runs both strategies repeatedly on identical scenario inputs. It alternates static and dynamic maps, applies deterministic route blockers, and records planner-only latency rather than UI rendering time.

Reference configuration

Setting	Value
Scenarios	10 seeded maps
Repetitions	3 per strategy
Grid	30 x 20
Obstacle density	20%
Dynamic blockers	Up to 2
Seed	42

Measured results

Metric	V1 Always-Replan	V2 Cached-Path
Runs	30	30
Success rate	100%	100%
Mean planning latency	2.382 ms	0.181 ms
Mean nodes explored	1,359	100
Mean path length	48	48
Mean planning calls	48	2
Mean replans	47	1

Measured improvement

On this local paired run, V2 reduced mean planning latency by 92.41%, explored nodes by 92.64%, and planning calls by 95.83%, while preserving the same path length and success rate. Absolute time varies by hardware and system load.

4. Phase 3 - Supervised learning

The supervised model learns to imitate A*. Instead of manually labeling actions, the generator creates random solvable mazes and converts each consecutive pair of A* path positions into one training example.

Action labels

```
class NavigationAction(IntEnum):
    UP = 0
    DOWN = 1
    LEFT = 2
    RIGHT = 3

@classmethod
def from_transition(cls, current, next_position):
    delta = (
        next_position[0] - current[0],
        next_position[1] - current[1],
    )
    for action in cls:
        if action.direction.value == delta:
            return action
    raise ValueError("Positions must be orthogonally adjacent")
```

Explicit integer values make the saved label mapping stable. The conversion checks that the teacher path moves exactly one orthogonal cell; malformed path data fails immediately.

Why randomized endpoints matter

Using only top-left starts and bottom-right goals would bias labels toward RIGHT and DOWN. The generator therefore chooses randomized, well-separated endpoints. A test confirms that generated samples include all four actions.

Observation encoding

Each state becomes three binary channels with shape (3, height, width). Separate channels prevent ambiguous numeric cell codes and map directly to CNN input channels.

Channel	Meaning	Active cells
0 - Obstacles	Blocked grid positions	Zero or more
1 - Agent	Current decision position	Exactly one
2 - Goal	Destination position	Exactly one

```
def encode_environment(environment):
    return (
        _binary_channel(environment.width, environment.height,
                        environment.obstacles),
        _binary_channel(environment.width, environment.height,
                        {environment.agent}),
        _binary_channel(environment.width, environment.height,
                        {environment.goal}),
    )
```

Nested tuples make observations immutable snapshots. Moving the live environment later cannot silently modify an earlier training example.

One maze becomes multiple samples

```
for step_index, (current, following) in enumerate(zip(path, path[1:])):
    action = NavigationAction.from_transition(current, following)
    samples.append(NavigationSample(
        maze_id=maze_id,
        step_index=step_index,
        observation=encode_environment(environment),
        action=action,
    ))
    environment.move(action.direction)
```

The observation is captured before movement, and the label describes that next movement. Then the agent advances so the following sample contains the new position.

Dataset generation and storage

A seeded random generator creates obstacles and endpoints. A* attempts to solve each maze. Unsolvable candidates are discarded, ensuring every included sample has a valid teacher action.

```
while len(labeled_mazes) < config.maze_count:
    maze_seed = generator.randrange(0, 2**63)
    environment = _create_random_environment(config, seed=maze_seed)
    result = planner.plan(environment, environment.start, environment.goal)
    if not result.success:
        continue
    samples = _label_path(environment, result.path, maze_id=maze_id)
    labeled_mazes.append(LabeledMaze(maze_id, samples))
```

Leakage-safe splitting

Whole mazes are assigned to train, validation, or test splits. Individual route steps are never shuffled across splits because adjacent steps from the same maze are nearly identical; splitting them would make evaluation misleadingly easy.

Generated dataset

Split	Mazes	Samples	Purpose
Train	80	1,332	Optimizer updates
Validation	10	182	Select the best epoch
Test	10	151	Untouched final evaluation

Files are gzip-compressed JSON: **train.json.gz**, **validation.json.gz**, and **test.json.gz**. The manifest records dimensions, density, seed, fractions, and counts. Gzip is storage compression; it does not change the JSON data.

Versioning

Dataset files include a schema version. Unsupported formats fail clearly instead of loading silently with the wrong assumptions.

PyTorch adapter and CNN

The saved dataset remains library-independent. A thin adapter converts a requested sample into PyTorch tensors, avoiding a second tensor copy of the entire dataset.

```
def __getitem__(self, index):
    sample = self._dataset.samples[index]
    observation = torch.tensor(sample.observation, dtype=torch.float32)
    action = torch.tensor(int(sample.action), dtype=torch.int64)
    return observation, action
```

A DataLoader batches observations from (3, 15, 20) into (batch, 3, 15, 20). Action labels use int64 because CrossEntropyLoss expects integer class indices.

Compact model

```
self.features = nn.Sequential(
    nn.Conv2d(3, 16, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.Conv2d(16, 32, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2),
    nn.AdaptiveAvgPool2d((3, 4)),
)
self.classifier = nn.Sequential(
    nn.Flatten(), nn.Linear(32 * 3 * 4, 64), nn.ReLU(), nn.Linear(64, 4)
)
```

The model has 29,988 trainable parameters. It returns four raw logits. Softmax is unnecessary for CrossEntropyLoss and unnecessary when selecting the largest score with argmax.

Training loop

The project uses an explicit PyTorch loop so every learning step remains visible rather than being hidden inside a high-level training framework.

```
for observations, actions in data_loader:
    observations = observations.to(device)
    actions = actions.to(device)

    optimizer.zero_grad(set_to_none=True)
    logits = model(observations)           # Forward propagation
    loss = loss_function(logits, actions)  # Cross-entropy error
    loss.backward()                        # Autograd + backpropagation
    optimizer.step()                       # Update model weights
```

Operation	Meaning
zero_grad	Remove gradients left from the previous batch
model(...)	Run convolutions and linear layers to produce logits
CrossEntropyLoss	Penalize low scores for the correct teacher action
backward	Calculate each parameter's contribution to the loss
Adam step	Adjust weights using gradients and optimizer state

Validation uses model.eval() and torch.inference_mode(), so it neither records gradients nor updates weights. The best validation epoch is copied and saved even if later epochs become worse.

Training configuration

Setting	Value
Epochs	20
Batch size	32
Learning rate	0.001
Optimizer	Adam
Seed	42
Device	CPU

Training results and checkpoints

The first real run trained on 1,332 samples and selected its checkpoint using 182 validation samples.

Metric	Measured value
Best epoch	11 of 20
Best validation action accuracy	63.7%
Epoch 20 training accuracy	69.8%
Epoch 20 validation accuracy	57.7%
Model parameters	29,988
Checkpoint size	Approximately 121 KB

What the curve suggests

Training accuracy continued upward while validation accuracy peaked and later declined. That gap is evidence of overfitting, so the saved model correctly uses epoch 11 rather than epoch 20.

Checkpoint contents

- Model configuration and best model_state_dict
- Epoch and validation accuracy
- Training hyperparameters and dataset manifest
- Stable action label mapping

The loader calls torch.load(..., weights_only=True), avoiding arbitrary pickled Python objects. It validates checkpoint version and action labels before constructing the model.

Generated artifacts

```
artifacts/models/cnn-v1/
|-- best_model.pt
|-- history.json
`-- training_config.json
```

Low-latency CNN inference

Interactive navigation should not rebuild every input channel or track gradients. PathPilot loads the checkpoint once and maintains a reusable observation tensor.

```
def encode(self, environment):
    if tensor_is_missing_or_grid_changed:
        self._initialize(environment)
    else:
        self._update_obstacles(environment)
        self._update_single_cell_channel(AGENT_CHANNEL,
                                         previous=self._agent,
                                         current=environment.agent)
        self._update_single_cell_channel(GOAL_CHANNEL,
                                         previous=self._goal,
                                         current=environment.goal)
    return self._tensor, perf_counter_ns() - started_at
```

Only changed obstacle cells are toggled. For agent movement, the old cell is cleared and the new cell is set. This preserves correctness for dynamic obstacles while reducing allocation and encoding work.

```
def predict_action(self, observation):
    if observation.ndim == 3:
        observation = observation.unsqueeze(0)
    self.eval()
    with torch.inference_mode():
        action_index = int(self(observation).argmax(dim=1).item())
    return NavigationAction(action_index)
```

Inference mode disables autograd bookkeeping. Argmax selects the highest logit directly, so a softmax pass is not needed. Encoding and model inference are timed separately to reveal the real bottleneck.

CPU first

For a 29,988-parameter model and batch size 1, CPU execution avoids device-transfer overhead. GPU or Apple MPS should be adopted only after a paired benchmark shows a benefit.

CNN controller and Pygame integration

The controller asks the learned policy for one action at a time, passes that action through the same GridEnvironment.move() validation used by humans and A*, and records the result.

```
prediction = self.policy.predict(environment)
self.metrics.inference_calls += 1
self.metrics.encoding_latency_ns += prediction.encoding_latency_ns
self.metrics.inference_latency_ns += prediction.inference_latency_ns

result = environment.move(prediction.action.direction)
if result not in {MoveResult.MOVED, MoveResult.REACHED_GOAL}:
    self.metrics.invalid_actions += 1
else:
    self.metrics.path_length += 1
```

Termination safeguards

- Invalid predictions cannot bypass collision or boundary validation.
- Repeated state-action decisions identify stuck or cyclic behavior.
- A maximum decision count prevents endless episodes.
- There is intentionally no hidden A* fallback, which keeps CNN evaluation honest.

UI behavior

The AI row contains Plan, V1, V2, and CNN controls. Pause and Stop share the existing lifecycle. Live CNN metrics show decisions, moves, invalid actions, cumulative encoding latency, and cumulative inference latency.

Optional dependency behavior

Manual and A* navigation still work without PyTorch. If ML dependencies or the checkpoint are missing, the CNN control shows an explanatory status instead of crashing the application.

What the first CNN smoke run taught us

A real checkpoint smoke run started from the default 20 x 15 environment. It verified loading, tensor updates, inference timing, movement validation, and safe termination. It was not the formal held-out evaluation.

Observation	Value
Final state	Failed safely
Agent / goal	(19, 0) / (19, 14)
Decisions	22
Successful moves	19
Invalid actions	3
Cumulative encoding time	0.199 ms
Cumulative inference time	4.309 ms

Core lesson

63.7% validation action accuracy did not produce dependable multi-step navigation. If each action has some chance of being wrong, errors compound across a long route. Episode success, collisions, loops, and path efficiency must be measured directly.

Likely causes to investigate

- Only 100 mazes and 1,332 training samples
- Teacher examples cover optimal path states but few recovery states after mistakes
- Class balance is improved but not perfectly uniform
- The compact network may underfit spatial navigation relationships
- Training objective rewards one-step imitation, not full episode completion

5. Testing and verification

The current repository has 107 passing unittest cases. Tests cover behavior rather than only import or syntax checks.

Area	Examples of verified behavior
Environment	Bounds, collisions, endpoints, obstacle toggling, reset semantics
A*	Shortest routes, failure, deterministic exploration, no mutation
Controllers	Pause, stop, replanning rules, caching, dynamic blockers
Benchmark	Paired scenarios, summaries, CSV and JSON output
Dataset	Shape, binary values, serialization, seeds, split isolation
CNN	Forward shapes, gradients, parameter limit, action prediction
Training	Weight updates, validation immutability, best state
Checkpoint	Identical restored logits and version validation
Inference	Cached tensor equality and obstacle removal
Learned controller	Success, invalid actions, loops, step limit, pause
UI	Grid geometry and non-overlapping controls

Run the suite

```
cd /Users/sharanyamishra/Desktop/PersonalProject/Atrisa
PYTHONPATH=src .venv/bin/python -m unittest discover -s tests -v
```

Additional checks include Python bytecode compilation, whitespace validation with git diff --check, checkpoint restoration, and a headless Pygame startup smoke test.

Reproduce the project

1. Install

```
python3 -m venv .venv
.venv/bin/python -m pip install -e ".[ml]"
```

2. Run the game

```
PYTHONPATH=src .venv/bin/python -m pathpilot.game
```

3. Generate the supervised dataset

```
PYTHONPATH=src .venv/bin/python -m pathpilot.learning.generator \
  --mazes 100 --width 20 --height 15 --density 0.20 \
  --seed 42 --output datasets/default
```

4. Train the CNN

```
PYTHONPATH=src .venv/bin/python -m pathpilot.learning.train_cli \
  --dataset datasets/default --epochs 20 --batch-size 32 \
  --learning-rate 0.001 --seed 42 --device cpu \
  --output artifacts/models/cnn-v1
```

5. Run the Phase 2 benchmark

```
PYTHONPATH=src .venv/bin/python -m pathpilot.evaluation.benchmark \
  --scenarios 10 --repetitions 3 --width 30 --height 20 \
  --density 0.20 --dynamic-obstacles 2 --seed 42 \
  --output benchmarks
```

Status and next milestone

Phase	Status	Delivered
Phase 1	Complete	Playable modular Pygame navigation game
Phase 2	Complete	A*, visualization, V1/V2 controllers, paired benchmarks
Phase 3	In progress	Dataset, CNN, training, checkpoint, inference, Pygame controller
Phase 4	Not started	Gymnasium and PPO reinforcement learning
Phase 5	Not started	Agent harness and MCP
Phase 6	Not started	Unified evaluations and dashboard

Immediate next milestone: held-out CNN evaluation

- Run CNN and A* on the untouched test mazes.
- Measure navigation success, collisions, loops, path efficiency, and failure reason.
- Measure median and tail encoding/inference latency after warm-up.
- Keep maps, seeds, limits, and hardware metadata reproducible.
- Use evidence to decide whether to expand the dataset, change the observation, or alter the model.

Project principle

The next change should be driven by measured failure modes. The current CNN is a valid baseline precisely because its limitations are visible, reproducible, and not hidden behind A* fallback behavior.

Key takeaways

- Modular boundaries allowed three kinds of agent to share one environment.
- Caching removed most unnecessary A* work without changing results.
- A* supplied reproducible teacher labels for supervised learning.
- A compact model achieved moderate step accuracy but weak episode reliability.
- Tests, manifests, histories, and metrics turn each phase into an auditable engineering experiment.